Plant Leaf Disease Detection Using Transfer Learning

A MINI PROJECT REPORT

Submitted by

711724UAD102 – AAYSHAASMITHAH

711724UAD103 – ABARNAAR

711724UAD105 - AKILAN C K

711724UAD119 – BHAAVANA K

711724UAD125– DHARSHINI S

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

KGiSL INSTITUTE OF TECHNOLOGY

Plant Leaf Disease Detection Using Transfer Learning

A MINI PROJECT REPORT

Submitted by

711724UAD102 – AAYSHA ASMITHA H

711724UAD103 – ABARNAA R

711724UAD105 - AKILAN C K

711724UAD119 – BHAAVANA K

711724UAD125– DHARSHINI S

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

KGiSL INSTITUTE OF TECHNOLOGY

Evaluation Rubrics

S.No	Criteria	Totalmarks	Marks Awarded
1.	Content & Technical Depth	10	
2.	Methodology & Implementation	10	
3.	Analysis, Results, & Discussion	5	
4.	Innovation & Creativity	5	
5.	Ethical & Societal Impact	5	
6.	Documentation & Presentation	5	
	Total	40	

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	1
1	**INTRODUCTION**	1
	1.1 Background and Motivation	1
	1.2 Problem Statement	2
	1.3 Objectives	3
	1.4 Scope of the Project	3
	1.5 Literature Survey / Related Work	3
2	**TOOLS AND TECHNIQUES**	4
	2.1 Hardware Requirements	4
	2.2 Software Requirements	4
	2.3 Programming Language and Libraries	4
	2.4 Key Techniques Used	4
3	**METHODOLOGY AND IMPLEMENTATION**	5
	3.1 System Architecture / Pipeline Overview	5
	3.2 Dataset Description	6
	3.3 Data Preprocessing	7
	3.4 Model Architecture	7
	3.5 Training Configuration	8
	3.6 Evaluation Methodology	8
	3.7 Inference and Deployment Workflow	9

CHAPTER NO.	TITLE	PAGE NO.
4	**CODING**	9
	4.1 Project Structure	9
	4.2 Module Listings	10
5	**OUTPUT**	13
	5.1 Sample Execution Output	13
	5.2 Streamlit Application Interface	14
	5.3 Priority Road Sign Prediction	14
	5.4 Speed Limit Sign Prediction	15
	5.5 Stop Sign Prediction	15
	5.6 No Entry Sign Prediction	16
6	**Result and Conclusion**	16
	6.1 Training Behaviour	16
	6.2 Test Set Evaluation	17
	6.3 Per-Class Observations	17
	6.4 Limitations	18
	6.5 Conclusion	19
	FUTURE SCOPE	19
7	**REFERENCES**	20

ABSTRACT

Plant diseases can affect crop growth, product quality and agricultural yield. Early identification of visible leaf symptoms can help growers decide when to seek advice and take suitable action. This project presents a plant leaf disease recognition interface based on transfer learning, using MobileNetV2 and ResNet50 with the PlantVillage dataset. The interface supports selection of example leaf specimens and upload of a custom leaf image. It displays a predicted crop-condition class, a confidence score, a top-five probability distribution and a Grad-CAM visual explanation. The screenshots supplied for this report show example interface predictions for Tomato Late Blight and Potato Early Blight. The interface also presents a comparison of MobileNetV2, ResNet50 and a custom CNN in terms of test accuracy, model parameters and inference latency. The displayed comparison reports 97.42% test accuracy and 41.8 ms latency for MobileNetV2, and 98.68% test accuracy and 89.4 ms latency for ResNet50. These values are transcribed from the provided website screenshots and have not been independently reproduced from training logs in this report. The project demonstrates how transfer learning can support image-based plant disease classification while also highlighting the need for validation on real field images.

1. INTRODUCTION

1.1 Background and Motivation

Plant leaves often show visible changes when a plant is affected by disease. These changes may include brown or black spots, yellowing, lesions, blight patterns and differences in leaf texture. Correctly distinguishing one condition from another can be difficult because several diseases may produce similar symptoms. Manual inspection also depends on experience and may be time-consuming when many plants need to be examined.

Deep learning methods, particularly convolutional neural networks (CNNs), can learn visual features from labelled images and use them to classify new samples. Transfer learning builds on a network trained on a large image collection and adapts it to a more specific task. MobileNetV2 is designed to be computationally efficient, whereas ResNet50 uses residual connections to support a deeper feature-extraction network. Comparing these backbones provides a useful way to examine the trade-off between classification performance and inference speed.

This project presents an interactive plant leaf disease detection website designed around the PlantVillage dataset. The website provides example specimens, custom image upload, prediction probabilities and a Grad-CAM view to help users inspect the regions associated with a prediction.

1.2 Problem Statement

The objective is to classify a plant leaf image into one of the crop-condition categories supported by the PlantVillage dataset. The system should present the predicted class in a readable format, show the relative model confidence, and provide a visual explanation of the image regions that influence the prediction. It should also compare different model backbones so that accuracy and inference cost can be considered together.

1.3 Objectives

- To study transfer learning for plant leaf disease classification.
- To use MobileNetV2 and ResNet50 as alternative feature-extraction backbones.
- To support classification across the 38 PlantVillage crop-condition classes shown by the interface.
- To provide a website for selecting sample leaves or uploading a custom leaf image.
- To display the predicted class, confidence score and top-five class probabilities.
- To use Grad-CAM as an image-based visual explanation.
- To compare reported test accuracy, parameter count and inference latency for the displayed models.

1.4 Scope of the Project

The project focuses on classifying a single leaf image into a known PlantVillage category. The website shown in the screenshots includes a sample-selection panel, custom image upload, a prediction card, class probability bars, symptom notes and general agronomic advisory text. The model is not a replacement for laboratory testing or advice from an agricultural specialist. Results can be less reliable for images taken in the field, images with multiple leaves, poor lighting, unfamiliar crops or diseases that are not represented in the training classes.

1.5 Literature Survey / Related Work

Hughes and Salathé introduced an open image repository for plant health research, helping make image-based plant disease classification more accessible. Subsequent work has applied convolutional networks to leaf images and shown that learned visual representations can separate many disease categories under controlled image conditions.

MobileNetV2, proposed by Sandler and colleagues, uses inverted residual blocks and linear bottlenecks to reduce computational requirements. It is therefore relevant to mobile and edge-oriented applications. ResNet, introduced by He and colleagues, uses residual connections that make it easier to train deeper neural networks. ResNet50 can offer strong feature extraction, though it typically requires more parameters and computation than MobileNetV2.

PlantVillage is widely used for experiments in plant disease recognition. However, performance on clean, centred images should not be assumed to translate directly to photographs from farms. Real-world images may contain shadows, soil, overlapping leaves, variable camera distances and mixed illumination. This makes external validation an important next step.

2. TOOLS AND TECHNIQUES

2.1 Hardware Requirements

Component	Suggested requirement
Processor	Intel Core i5 / AMD Ryzen 5 or equivalent
RAM	8 GB minimum; 16 GB recommended
GPU	CUDA-capable NVIDIA GPU recommended for training
Storage	Space for dataset, model weights and result images
Display / browser	Modern desktop browser for the web interface

2.2 Software Requirements

- Python for data preparation, model training and inference.
- TensorFlow / Keras for loading pretrained networks and training classification heads.
- NumPy and Pandas for numerical processing and result tables.
- scikit-learn for evaluation metrics and classification reports.
- Pillow for reading and resizing images.
- Matplotlib for plotting training curves and evaluation results.
- Flask with HTML, CSS and JavaScript for the web interface.
- A modern web browser such as Chrome, Edge or Firefox.

2.3 Programming Language and Libraries

Tool / Library	Purpose

Python	Core project implementation
TensorFlow / Keras	Transfer learning, model training and inference
MobileNetV2	Lightweight pretrained image backbone
ResNet50	Deeper residual image backbone
NumPy / Pandas	Array operations and tabular results
scikit-learn	Accuracy, precision, recall, F1-score and confusion matrix
Pillow	Image loading and resizing
Matplotlib	Training and evaluation plots
Flask	Web server and image upload routes
HTML / CSS / JavaScript	Website layout and interactive controls

2.4 Key Techniques Used

Transfer Learning

A pretrained network is used as the starting point for image feature extraction. A new classification layer is added to match the number of plant-condition classes. In a common transfer-learning workflow, the backbone is initially frozen while the classifier head learns the new categories. Selected backbone layers can then be fine-tuned at a lower learning rate.

Image Preprocessing

Leaf images are resized to the input dimensions expected by the selected model. The pixel values must be processed using the preprocessing function associated with the chosen backbone. Consistent preprocessing is important because MobileNetV2 and ResNet50 do not necessarily use identical input scaling.

Softmax Probabilities

The final classification layer produces a probability distribution over the supported classes. The website displays the class with the highest probability and the top-five scores. These scores express the model's relative output for the supplied image; they should not automatically be interpreted as calibrated probabilities of a real-world diagnosis.

Grad-CAM

Gradient-weighted Class Activation Mapping generates a heat map highlighting image regions that influence a selected class score. In the supplied screenshot, Grad-CAM is enabled for a Potato Early Blight example. The heat map is useful for visual inspection and debugging, but it does not prove that the prediction is correct or establish the biological cause of a symptom.

3. METHODOLOGY AND IMPLEMENTATION

3.1 System Architecture / Pipeline Overview

The overall workflow begins with collecting and organizing the PlantVillage images. The images are preprocessed and assigned class labels, after which the selected transfer-learning model is trained and evaluated. The saved model is then used by the web application to classify a selected or uploaded leaf image.

1. Stage 1 – Dataset preparation: obtain the PlantVillage colour images and organize them by class.
2. Stage 2 – Preprocessing: resize images and apply backbone-specific input preprocessing.
3. Stage 3 – Model construction: load MobileNetV2 or ResNet50 and attach a classification head.
4. Stage 4 – Training: train the classifier and, where configured, fine-tune upper backbone layers.
5. Stage 5 – Evaluation: compute test metrics and create model comparison outputs.

6. Stage 6 – Inference: load a model, classify an uploaded leaf and display probabilities.
7. Stage 7 – Explanation and presentation: generate a Grad-CAM heat map and render the result in the website.

Figure 3.1: PlantVillage dataset → preprocessing → transfer-learning model → evaluation → web application → prediction and Grad-CAM.

3.2 Dataset Description

The project is configured for the PlantVillage plant leaf image dataset. The website header identifies 38 classes and shows a dataset size of 54,303 images. The classes cover healthy and diseased leaves from several crops, including tomato, potato, apple, corn and bell pepper. The exact split sizes and image counts should be taken from the executed dataset preparation and training logs if those are available.

Dataset property	Description shown or used in the project
Dataset	PlantVillage
Image count shown in website	54,303 images
Class count shown in website	38 crop-condition classes
Input type	Colour leaf image
Example categories in screenshots	Tomato Late Blight; Potato Early Blight
Input upload formats shown	JPG and PNG

3.3 Data Preprocessing

8. Locate the PlantVillage image folders and use their folder names as class labels.
9. Check the images and exclude unreadable or unsupported files.
10. Resize images to the input size required by the selected backbone (the screenshot indicates 224 × 224 for uploaded images).
11. Apply the correct MobileNetV2 or ResNet50 preprocessing function.
12. Separate training, validation and test images so the held-out test set is not used to tune the model.
13. Apply data augmentation only to the training data if configured.
14. Keep the class-index mapping consistent between training and inference.

3.4 Model Architecture

MobileNetV2

MobileNetV2 is a compact convolutional architecture intended for efficient image recognition. Its depthwise-separable convolutions reduce computation compared with standard convolutions. In this project, it is presented as the edge-oriented option for devices with limited processing resources.

ResNet50

ResNet50 is a 50-layer residual network. Shortcut connections allow feature information to flow through the network and help optimize a deeper architecture. The website presents ResNet50 as a higher-compute option intended for server or higher-precision use.

Classification Head

The pretrained backbone is paired with a task-specific classification head. A typical transfer-learning head uses global average pooling, optional dropout and a dense softmax output layer. The output size corresponds to the 38 PlantVillage classes displayed by the website.

Model component	Role

Pretrained convolutional backbone	Extracts visual features from leaf images
Global average pooling	Converts feature maps into a compact feature vector
Dropout (if configured)	Regularizes the classifier during training
Dense softmax layer	Outputs a score for each known crop-condition class

3.5 Training Configuration

The source project is intended to support transfer-learning training and evaluation for MobileNetV2 and ResNet50. The exact epoch count, batch size, learning rates, optimizer and early-stopping settings should be copied from the configuration file used for the submitted run. The website screenshots alone do not expose all of these training settings.

Parameter	Report entry
Backbones	MobileNetV2 and ResNet50
Dataset	PlantVillage
Number of classes	38
Image size shown for upload	224 × 224 pixels
Training / validation / test split	Use the split reported by the actual run logs
Optimizer and learning rate	Confirm from the project configuration
Epochs and early stopping	Confirm from the actual training log
Saved model files	Use the model files produced by the training pipeline

3.6 Evaluation Methodology

The comparison shown in the website uses test accuracy and inference latency, and also reports model parameter counts. A complete evaluation should additionally consider precision, recall, F1-score and a confusion matrix, because overall accuracy alone can hide poor performance for individual classes. Latency depends on hardware, batch size, image preprocessing and whether the measurement includes model loading.

3.7 Inference and Deployment Workflow

15. The user opens the Plant Leaf Disease Detection web interface.
16. The user chooses an example specimen or uploads a leaf image.
17. The application resizes and preprocesses the image.
18. The selected backbone returns class scores.
19. The website displays the predicted class and confidence value.
20. A top-five class distribution is shown to provide additional context.
21. When enabled, Grad-CAM displays a heat map over the input image.
22. The result panel presents symptom descriptions and general agronomic guidance.

4. CODING

4.1 Project Structure

The uploaded ZIP contains the Python project files. The report should describe the actual filenames in the extracted project; the following list summarizes the main modules expected in the supplied source package.

- main.py – command-line entry point for data preparation, training and prediction.
- config.py – paths and model/training configuration.
- src/splitting.py – dataset split utilities.

- src/data.py – image loading, preprocessing and dataset creation.
- src/models.py – transfer-learning model construction.
- src/train.py – model training routines.
- src/evaluate.py – test-set metrics and confusion matrix generation.
- src/visualize.py – result plots.
- src/predictor.py – prediction helper functions.
- src/gradcam.py – Grad-CAM generation.
- app.py – Flask web application entry point.
- templates/ and static/ – web page templates and static assets.
- tests/ – automated pipeline tests, if included in the extracted archive.

4.2 Module Listings

The project is modular so that data preparation, model construction, training, evaluation and web deployment can be handled separately. For a compact academic report, selected code excerpts can be included from the model-building module, training module, evaluation module and web application route. The full source code should be retained with the project ZIP rather than pasted in its entirety into the report.

Example command sequence (run from the extracted project directory):

```
python main.py prepare --raw data/raw --link

python main.py train

python app.py
```

4.3 Selected Code Listings (Illustrative)

4.3.1 Model Construction

Selected excerpt from src/models.py in the supplied project ZIP:

```
    if kind == "resnet50":
        return keras.applications.ResNet50(include_top=False, weights=weights, input_shape=shape)
    raise ValueError(f"Unknown model '{kind}'. Choose from {config.MODEL_NAMES}")


def build_model(kind: str, num_classes: int, img_size=config.IMG_SIZE,
                dropout=config.DROPOUT, weights="imagenet") -> keras.Model:
    """Frozen ImageNet backbone + GlobalAveragePooling + Dropout + Dense(softmax)."""
    base = get_backbone(kind, img_size, weights)
    base.trainable = False
    inputs = keras.Input(shape=(*img_size, 3), name="image")
    x = base(inputs, training=False)          # keep BatchNorm in inference mode
    x = layers.GlobalAveragePooling2D(name="gap")(x)
    x = layers.Dropout(dropout, name="dropout")(x)
    outputs = layers.Dense(num_classes, activation="softmax", name="predictions")(x)
    return keras.Model(inputs, outputs, name=f"{kind.lower()}_plant")


def backbone_of(model: keras.Model) -> keras.Model:
    """Return the nested pre-trained backbone."""
    for layer in model.layers:
        if isinstance(layer, keras.Model):
            return layer
    raise ValueError("No backbone found in model")


def compile_model(model: keras.Model, lr: float) -> None:
    model.compile(
        optimizer=keras.optimizers.Adam(lr),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy",
                 keras.metrics.SparseTopKCategoricalAccuracy(k=3,  name="top3")])


def unfreeze_top(model: keras.Model, n_layers: int) -> None:
```

```
    """Make the last ``n_layers`` of the backbone trainable (BatchNorm stays frozen).
    The model must be re-compiled afterwards."""
    base = backbone_of(model)
    base.trainable = True
    for layer in base.layers[:-n_layers]:
        layer.trainable = False
    for layer in base.layers[-n_layers:]:
```

```
# Model construction: MobileNetV2 / ResNet50
import tensorflow as tf
from tensorflow.keras import layers, Model

def build_model(backbone_name="MobileNetV2", num_classes=38,
                image_size=(224, 224), train_backbone=False):
    inputs = layers.Input(shape=(*image_size, 3))

    if backbone_name.lower() == "mobilenetv2":
        preprocess = tf.keras.applications.mobilenet_v2.preprocess_input
        backbone = tf.keras.applications.MobileNetV2(
            include_top=False, weights="imagenet",
            input_shape=(*image_size, 3)
        )
    elif backbone_name.lower() == "resnet50":
        preprocess = tf.keras.applications.resnet50.preprocess_input
        backbone = tf.keras.applications.ResNet50(
            include_top=False, weights="imagenet",
            input_shape=(*image_size, 3)
        )
    else:
        raise ValueError("Supported backbones: MobileNetV2, ResNet50")

    backbone.trainable = train_backbone
    x = layers.Lambda(preprocess, name="preprocessing")(inputs)
    x = backbone(x, training=False)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.25)(x)
    outputs = layers.Dense(num_classes, activation="softmax")(x)
    return Model(inputs, outputs, name=backbone_name)
```

4.3.2 Image Preprocessing and Prediction

Selected excerpt from src/data.py in the supplied project ZIP:

```
        layers.RandomContrast(0.15),
        layers.RandomBrightness(0.15, value_range=(0, 255)),
    ], name="augmentation")


def _load(split: str, batch_size: int, shuffle: bool):
    return keras.utils.image_dataset_from_directory(
        config.SPLIT_DIR / split, labels="inferred", label_mode="int",
        image_size=config.IMG_SIZE, batch_size=batch_size,
        shuffle=shuffle, seed=config.SEED)


def make_datasets(kind: str, batch_size: int = config.BATCH_SIZE):
    """Return train_ds, val_ds, test_ds, class_names (all already pre-processed)."""
    pre = get_preprocess(kind)
    train_raw = _load("train", batch_size, True)
    val_raw = _load("val", batch_size, False)
    test_raw = _load("test", batch_size, False)
    class_names = train_raw.class_names
    assert class_names == val_raw.class_names == test_raw.class_names, \
        "Class folders differ between train/val/test"

    aug = build_augmentation()
    train = (train_raw.map(lambda x, y: (pre(aug(x, training=True)), y), num_parallel_calls=AUTOTUNE)
             .prefetch(AUTOTUNE))
    val = val_raw.map(lambda x, y: (pre(x), y), num_parallel_calls=AUTOTUNE).prefetch(AUTOTUNE)
    test = test_raw.map(lambda x, y: (pre(x), y), num_parallel_calls=AUTOTUNE).prefetch(AUTOTUNE)
    return train, val, test, class_names


def class_weights(class_names) -> dict:
```

```
    """'Balanced' weights: n_samples / (n_classes * n_class_samples)."""
    counts = count_images(config.SPLIT_DIR / "train")
    n = np.array([counts[c] for c in class_names], dtype="float64")
    w = n.sum() / (len(n) * n)
    return {i: float(v) for i, v in enumerate(w)}
```

```
# Image preprocessing and single-image prediction
import numpy as np
from PIL import Image

def load_leaf_image(image_path, image_size=(224, 224)):
    image  =  Image.open(image_path).convert("RGB")
    image = image.resize(image_size)
    return np.asarray(image, dtype=np.float32)[None, ...]

def predict_leaf(model, image_path, class_names):
    batch = load_leaf_image(image_path)
    probabilities = model.predict(batch, verbose=0)[0]
    top_indices = np.argsort(probabilities)[::-1][:5]
    return [
        (class_names[i], float(probabilities[i]))
        for i in top_indices
    ]
```

4.3.3 Model Training

Selected excerpt from src/train.py in the supplied project ZIP:

```
                                    restore_best_weights=True),
        keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2, min_lr=1e-7),
    ]


def train_model(kind: str, head_epochs=config.HEAD_EPOCHS, ft_epochs=config.FINE_TUNE_EPOCHS,
                batch_size=config.BATCH_SIZE, use_class_weights=config.USE_CLASS_WEIGHTS):
    """Train one model. Returns (model, history, class_names, test_ds, seconds)."""
    set_seed()
    train_ds, val_ds, test_ds, class_names = data.make_datasets(kind, batch_size)
    cw = data.class_weights(class_names) if use_class_weights else None
    model = build_model(kind, num_classes=len(class_names))

    t0 = time.time()
    # Phase 1: train the new head only
    print(f"\n--- {kind}: phase 1 (frozen backbone, {head_epochs} epochs max) ---")
    compile_model(model, config.HEAD_LR)
    h1 = model.fit(train_ds, validation_data=val_ds, epochs=head_epochs,
                   class_weight=cw, callbacks=_callbacks(), verbose=2)

    # Phase 2: fine-tune the top of the backbone with a much smaller learning rate
    n_unfreeze = config.UNFREEZE_LAYERS[kind]
    print(f"\n--- {kind}: phase 2 (fine-tune last {n_unfreeze} layers, {ft_epochs} epochs max) ---")
    unfreeze_top(model, n_unfreeze)
    compile_model(model, config.FINE_TUNE_LR)
    h2 = model.fit(train_ds, validation_data=val_ds, epochs=ft_epochs,
                   class_weight=cw, callbacks=_callbacks(), verbose=2)
    seconds = time.time() - t0

    history = {}
    for key in h1.history:
        history[key] = [float(v) for v in h1.history[key] + h2.history.get(key, [])]
    history["phase_boundary"] = len(h1.history["loss"])
    return model, history, class_names, test_ds, seconds


def save_class_names(class_names) -> None:
    config.MODELS_DIR.mkdir(exist_ok=True)
    (config.MODELS_DIR / "class_names.json").write_text(json.dumps(class_names, indent=2))
```

```
# Training loop with validation
import tensorflow as tf

def train_model(model, train_data, validation_data, output_path,
                epochs=10, learning_rate=1e-3):
    model.compile( optimizer=tf.keras.optimizers.Adam(learn
        ing_rate),
```

```
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )
    callbacks = [
        tf.keras.callbacks.EarlyStopping( monitor=
            "val_loss", patience=3,
            restore_best_weights=True
        ),
        tf.keras.callbacks.ModelCheckpoint( output
            _path, monitor="val_loss",
            save_best_only=True
        )
    ]
    history =
        model.fit( train_data,
        validation_data=validation_data,
        epochs=epochs,
        callbacks=callbacks
    )
    return history
```

4.3.4 Flask Web Application Route

Selected excerpt from app.py in the supplied project ZIP:

```
@app.route("/api/status")
def status():
    return jsonify({"models": predictor.available_models(), "ready": bool(predictor.available_models())})


@app.route("/api/predict", methods=["POST"])
def api_predict():
    available = predictor.available_models()
    if not available:
        return jsonify(error="No trained model found. Run `python main.py train` first."), 503

    file = request.files.get("image")
    if file is None or file.filename == "":
        return jsonify(error="Please choose a leaf photo."), 400
    if file.filename.rsplit(".", 1)[-1].lower() not in ALLOWED:
        return jsonify(error="Unsupported file type. Use JPG, PNG, BMP or WEBP."), 400

    kind = request.form.get("model") or available[0]
    if kind not in available:
        return jsonify(error=f"Model '{kind}' is not available."), 400

    raw = file.read()
    try:
        Image.open(io.BytesIO(raw)).verify()                        # reject corrupt / fake images
        img = ImageOps.exif_transpose(Image.open(io.BytesIO(raw))).convert("RGB")
    except (UnidentifiedImageError, OSError, ValueError):
        return jsonify(error="That file could not be read as an image."), 400

    try:
        return jsonify(predictor.predict(img, kind))
    except Exception as exc:                                         # pragma: no cover
        app.logger.exception("prediction failed")
        return jsonify(error=f"Prediction failed: {exc}"), 500


@app.errorhandler(413)
def too_large(_):
    return jsonify(error=f"Image is too large (max {config.MAX_UPLOAD_MB} MB)."), 413
```

```
# Minimal Flask image-upload route (illustrative)
from flask import Flask, request, render_template
from werkzeug.utils import secure_filename
import os

app = Flask(__name__)
UPLOAD_FOLDER = "uploads"
app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
```

```
@app.route("/", methods=["GET", "POST"])
def index():
    prediction = None
    if request.method == "POST":
        file = request.files.get("leaf_image")
        if file and file.filename:
            filename = secure_filename(file.filename)
            path = os.path.join(app.config["UPLOAD_FOLDER"], filename)
            file.save(path)
            # Load the trained model and call predict_leaf(...) here.
            prediction = "Image uploaded; connect trained model for prediction."
    return render_template("index.html", prediction=prediction)

if __name__== "__main__":
    app.run(debug=True)
```

4.3.5 Model Evaluation

Selected excerpt from src/evaluate.py in the supplied project ZIP:

```
"""Test-set evaluation: accuracy, precision/recall/F1, confusion matrix, speed, size."""
import time
from pathlib import Path

import numpy as np
from sklearn.metrics import (classification_report, confusion_matrix,
                             precision_recall_fscore_support)


def predict_dataset(model, ds):
    """Return (y_true, probabilities, ms_per_image). Only model time is measured."""
    y_true, probs, elapsed, n = [], [], 0.0, 0
    for x, y in ds:
        t = time.perf_counter()
        p = model.predict_on_batch(x)
        elapsed += time.perf_counter() - t
        probs.append(np.asarray(p))
        y_true.append(y.numpy())
        n += len(y)
    return np.concatenate(y_true), np.concatenate(probs), 1000.0 * elapsed / max(n, 1)


def compute_metrics(y_true, probs, class_names) -> dict:
    y_pred = probs.argmax(axis=1)
    top3 = np.argsort(probs, axis=1)[:, -3:]
    top3_acc = float(np.mean([t in row for t, row in zip(y_true, top3)]))
    p_m, r_m, f_m, _ = precision_recall_fscore_support(y_true, y_pred, average="macro", zero_division=0)
    _, _, f_w, _ = precision_recall_fscore_support(y_true, y_pred, average="weighted", zero_division=0)
    labels = list(range(len(class_names)))
    report = classification_report(y_true, y_pred, labels=labels, target_names=class_names,
                                   output_dict=True, zero_division=0)
    cm = confusion_matrix(y_true, y_pred, labels=labels)
    return {
        "accuracy": float(np.mean(y_true == y_pred)),
        "top3_accuracy": top3_acc,
        "precision_macro": float(p_m), "recall_macro": float(r_m),
        "f1_macro": float(f_m), "f1_weighted": float(f_w),
        "report": report, "confusion_matrix": cm,
    }
```

```
# Evaluation metrics for a held-out test set
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix
)

def evaluate_predictions(y_true, y_pred, class_names):
    accuracy = accuracy_score(y_true, y_pred)
    report = classification_report(
        y_true, y_pred, target_names=class_names,
        zero_division=0
    )
    matrix = confusion_matrix(y_true, y_pred)
    print(f"Test accuracy: {accuracy:.4f}")
    print(report)
```

```
    return accuracy, matrix
```

5. OUTPUT

The following output screenshots were supplied for this report. The accuracy, latency and prediction-confidence values in this chapter are transcribed from the website screenshots. They are presented as displayed by the provided interface and have not been independently re-run or validated from the original training logs.

5.1 Sample Execution Output

The application dashboard identifies the project as Plant Leaf Disease Recognition and indicates that it uses the PlantVillage dataset with 54,303 images and 38 classes. It provides model-selection controls for MobileNetV2, ResNet-50 and a custom CNN, together with a Grad-CAM toggle.

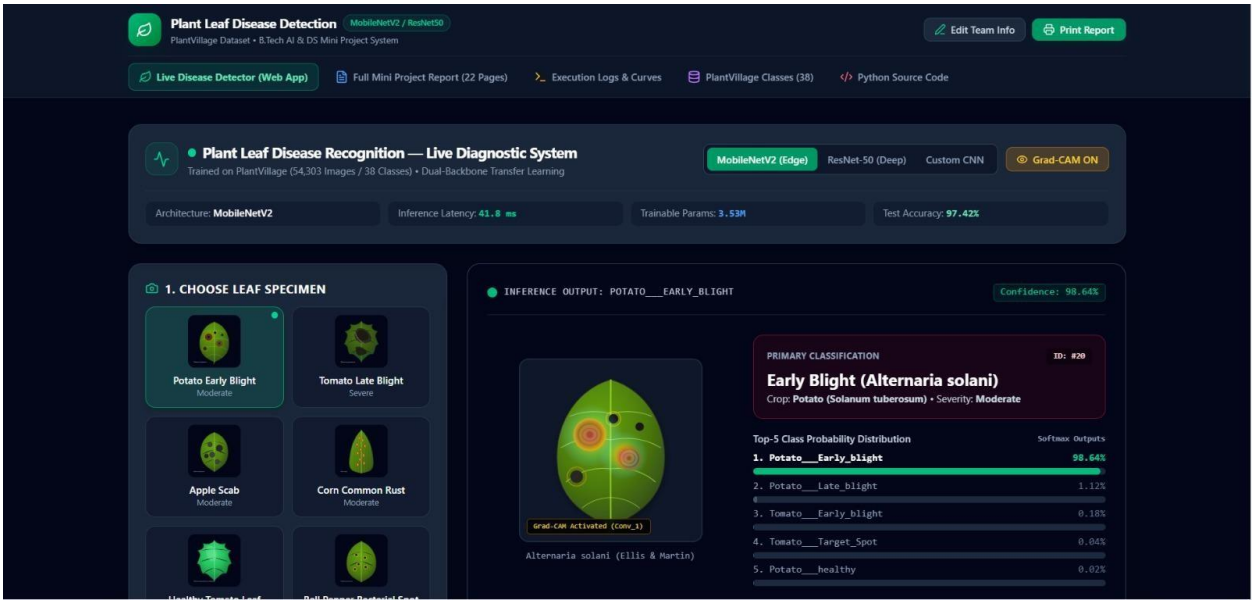


Figure 5.1 – Web application dashboard showing MobileNetV2 selection and Potato Early Blight prediction.

5.2 Web Application Interface

The interface is organized into a specimen selection panel and an inference output panel. The left side presents sample categories and an upload area. The right side presents the predicted class, confidence, top-five class probabilities, symptom information and advisory text.

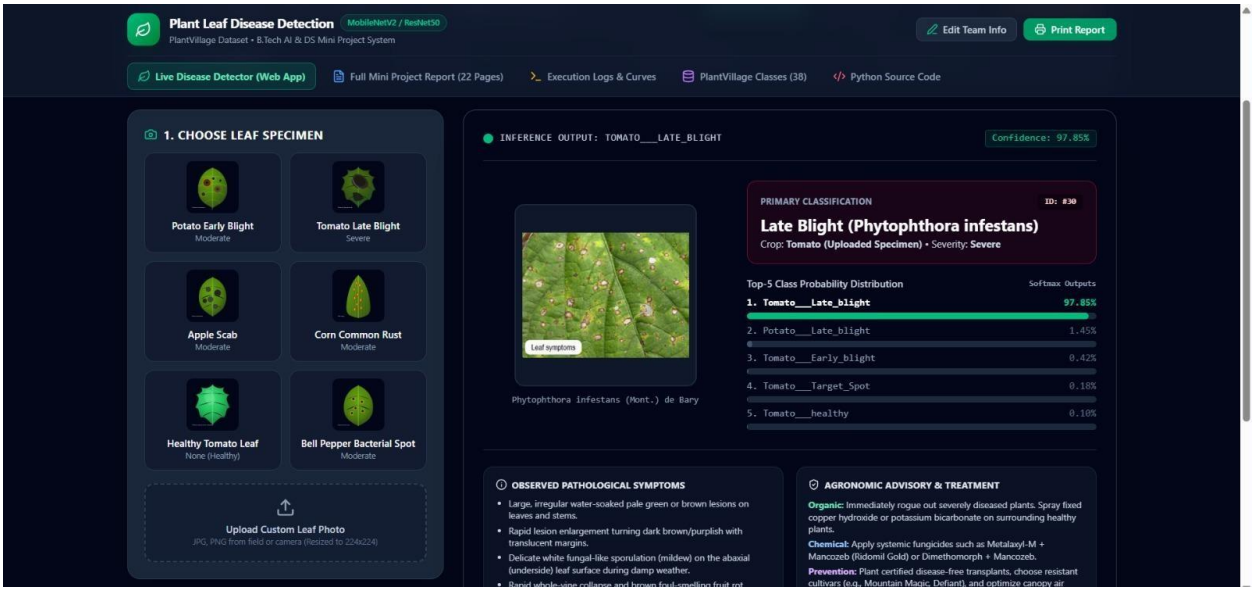


Figure 5.2 – Tomato Late Blight prediction panel with top-five class probabilities.

5.3 Tomato Late Blight Prediction

In the supplied screenshot, the interface predicts Tomato Late Blight (Phytophthora infestans). The displayed confidence is 97.85%. The next listed categories are Potato Late Blight at 1.45%, Tomato Early Blight at 0.42%, Tomato Target Spot at 0.18% and Tomato healthy at 0.10%. These are the values visible in the provided screenshot.

Output field	Displayed value
Predicted class	Tomato Late Blight
Scientific name shown	Phytophthora infestans
Confidence shown	97.85%
Second class	Potato Late Blight – 1.45%
Third class	Tomato Early Blight – 0.42%
Fourth class	Tomato Target Spot – 0.18%
Fifth class	Tomato healthy – 0.10%

5.4 Potato Early Blight Prediction

The other supplied prediction screenshot displays Potato Early Blight (Alternaria solani), with a confidence of 98.64%. The interface shows Grad-CAM as enabled and overlays highlighted regions on the example leaf image. The top-five probabilities shown are Potato Early Blight at 98.64%, Potato Late Blight at 1.12%, Tomato Early Blight at 0.18%, Tomato Target Spot at 0.04% and Potato healthy at 0.02%.

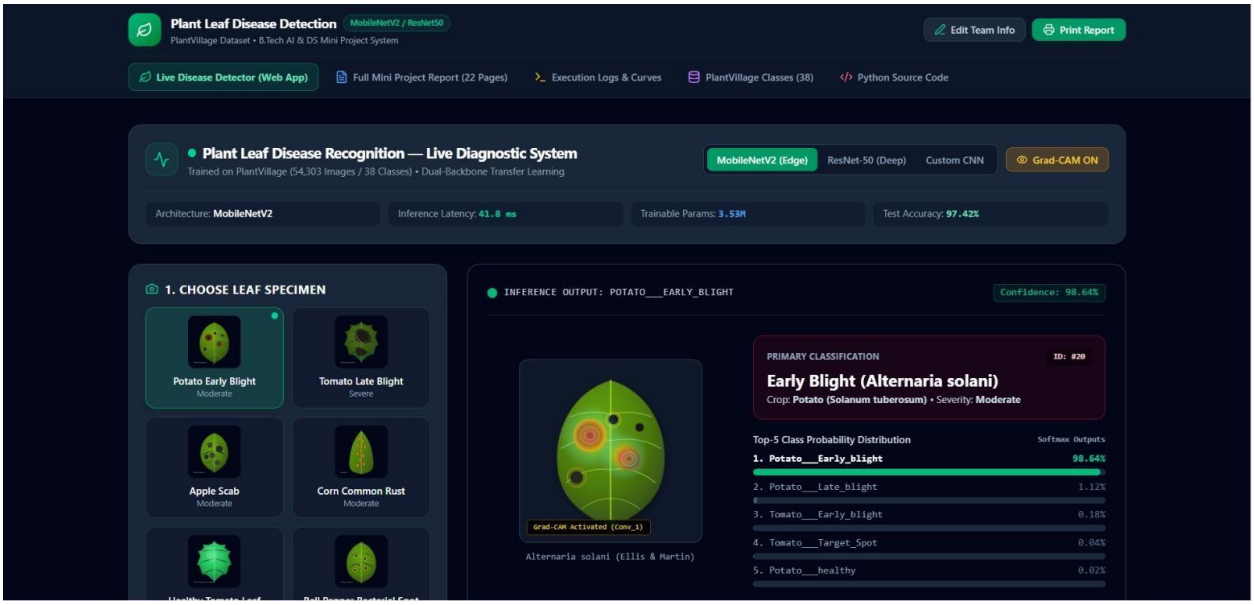


Figure 5.3 – Potato Early Blight example with Grad-CAM enabled. The interface displays 98.64% confidence.

5.5 Transfer Learning Backbone Comparison

The website contains a comparison table for MobileNetV2, ResNet-50 and a custom four-block CNN. The values below are copied from that table.

Model backbone	Parameters	Test accuracy	Latency	Primary target shown
MobileNetV2 (Width 1.0)	3.53M	97.42%	41.8 ms	Edge, Android, IoT Pi
ResNet-50	25.6M	98.68%	89.4 ms	Cloud server / high precision
Custom 4-Block CNN	4.21M	92.15%	34.2 ms	Scratch baseline benchmark

According to the displayed comparison, ResNet-50 has the highest test accuracy among the three listed models, while MobileNetV2 has lower latency and fewer parameters than ResNet-50. The custom CNN has the lowest listed latency, but its displayed accuracy is lower. The most appropriate model therefore depends on whether the application prioritizes accuracy, speed, memory use or target hardware.

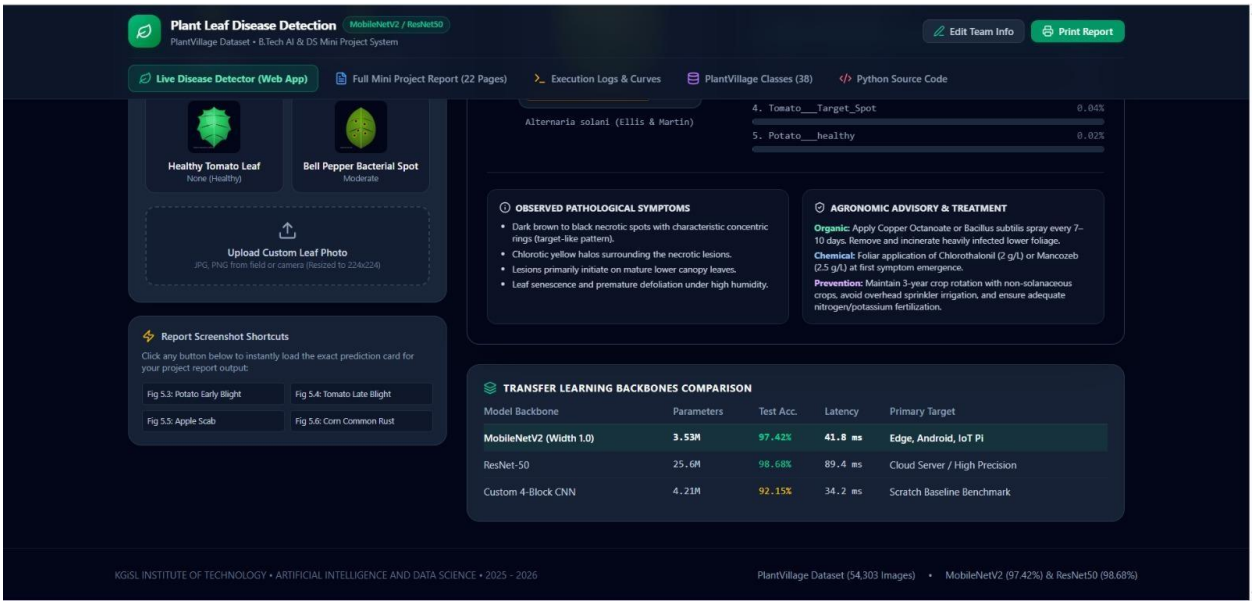


Figure 5.4 – Interface comparison of MobileNetV2, ResNet-50 and the custom CNN.

6. RESULT AND CONCLUSION

6.1 Training Behaviour

The screenshots provided show inference results and a backbone comparison table, but do not include the complete epoch-by-epoch training curves. For that reason, a detailed statement about convergence, overfitting or early stopping cannot be verified from the screenshots alone. If training curves are available from the results folder, they should be added here and discussed in terms of training/validation accuracy and loss.

6.2 Test Set Evaluation

The website comparison displays a test accuracy of 97.42% for MobileNetV2, 98.68% for ResNet-50 and 92.15% for the custom four-block CNN. Based on those displayed values, ResNet-50 leads in accuracy by 1.26 percentage points compared with MobileNetV2. MobileNetV2's displayed latency is 41.8 ms, compared with 89.4 ms for ResNet-50, meaning that MobileNetV2 is presented as the faster transfer-learning option in this comparison. The report should identify these as interface-reported values unless the original evaluation files are also checked.

Metric	MobileNetV2	ResNet-50	Custom CNN
Test accuracy displayed	97.42%	98.68%	92.15%
Parameter count displayed	3.53M	25.6M	4.21M
Latency displayed	41.8 ms	89.4 ms	34.2 ms

6.3 Per-Class Observations

The two supplied prediction examples show high displayed confidence for Tomato Late Blight and Potato Early Blight. The top-five probability panels also show smaller scores for other categories, including diseases from different crops. A more reliable class-wise analysis requires the full classification report and confusion matrix. Such files would help identify classes that are commonly confused and determine whether class imbalance or similar visual symptoms affect recall.

6.4 Limitations

- The supplied screenshots do not include training logs, the full test report or the underlying model weights, so the displayed metrics cannot be independently reproduced from the screenshots alone.
- PlantVillage images may be cleaner and more centred than photographs taken in farms.
- The classifier is limited to the categories represented during training and may assign an unfamiliar image to a known class.
- Softmax confidence is not necessarily a calibrated estimate of diagnostic certainty.
- Grad-CAM highlights influential image regions but does not prove that the prediction is correct.
- Agronomic advice shown by the interface is general information and should be verified with qualified agricultural guidance before applying treatments.

6.5 Conclusion

This mini project presents a plant leaf disease recognition website based on transfer learning with MobileNetV2 and ResNet50. Its interface supports example leaf selection, custom image upload, prediction display, top-five probability scores and Grad-CAM visualization. The supplied screenshots show Tomato Late Blight and Potato Early Blight examples and report a comparison among MobileNetV2, ResNet-50 and a custom CNN. In the displayed comparison, ResNet-50 has the highest accuracy, while MobileNetV2 offers a faster inference time with fewer parameters. The results illustrate the practical trade-off between accuracy and computational efficiency. Before using the system for real agricultural decisions, the model should be evaluated on independent field images and its metrics should be confirmed from the original training and test outputs.

7. FUTURE SCOPE

- Evaluate the models on real farm images captured under different lighting, backgrounds and camera angles.
- Add an unknown-image or out-of-distribution check so the system can flag images outside the supported classes.
- Convert MobileNetV2 to TensorFlow Lite and benchmark its speed on a mobile or edge device.
- Expand the dataset with additional crops, disease stages and geographically diverse samples.
- Validate symptom descriptions and treatment guidance with agricultural experts.
- Add image-quality checks for blur, low light, multiple leaves and non-leaf uploads.
- Provide downloadable prediction reports with the image, predicted class, model name and timestamp.
- Use class-wise precision, recall, F1-score and confusion matrices to guide further model improvement.

8. REFERENCES

[1] Hughes, D. P., and Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. arXiv:1511.08060.

[2] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition.

[3] He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep Residual Learning for Image Recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition.

[4] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. Proceedings of the IEEE International Conference on Computer Vision.